

O artigo Design by Contract, apresenta uma abordagem profunda e inovadora para a resolução de um dos problemas mais persistentes da engenharia de software: como construir sistemas que sejam simultaneamente corretos e robustos. Inserido no contexto da programação orientada a objetos e fundamentado na linguagem Eiffel, o autor propõe uma metáfora central: a construção de software deve ser vista como uma sucessão de decisões de contratação devidamente documentadas. Em vez de depender apenas de testes ou de uma arquitetura flexível, Meyer diz que a confiabilidade exige uma formalização rigorosa das responsabilidades de cada componente do sistema.

A Metáfora do Contrato e as Asserções

No coração da teoria de Meyer está a ideia de que a interação entre diferentes partes de um software (como a chamada de uma rotina) é semelhante a um contrato no mundo real, que carrega obrigações e benefícios mútuos para ambas as partes envolvidas. Em software, essas partes são o "cliente" (a rotina que faz a chamada) e o "fornecedor" (a rotina que é chamada).

Esses contratos são expressos através de mecanismos chamados asserções, que se dividem principalmente em três categorias:

Pré-condições (require): Representam as obrigações do cliente e os benefícios do fornecedor. O cliente deve garantir que certas condições sejam verdadeiras antes de chamar a rotina. Se a pré-condição for violada, o fornecedor não tem qualquer obrigação de executar a tarefa.

Pós-condições (ensure): Representam as obrigações do fornecedor e os benefícios do cliente. É a garantia de que, se a pré-condição foi respeitada, o fornecedor entregará o resultado esperado ao final da execução.

Invariantes de Classe: São restrições globais que definem a consistência de um objeto. Um invariante deve ser verdadeiro após a criação do objeto e deve ser mantido e preservado por todas as rotinas exportadas (públicas) da classe. O invariante age como uma cláusula geral que se aplica a todos os contratos daquela classe.

A Rejeição da "Programação Defensiva"

Um dos pontos mais polêmicos e impactantes do texto é a forte crítica de Meyer à chamada "programação defensiva". Tradicionalmente, os programadores são ensinados a proteger seus módulos com o máximo de verificações possíveis, tratando qualquer anomalia de forma redundante (o cliente verifica o erro, e o fornecedor verifica novamente).

Meyer argumenta que essa redundância não apenas aumenta drasticamente a complexidade do código, como também diminui a sua confiabilidade. Com a abordagem baseada em contratos, a responsabilidade é atribuída a apenas uma das partes. Se uma regra está na pré-condição, é um problema do cliente garantir

que ela seja cumprida; o fornecedor pode e deve focar exclusivamente em executar a sua tarefa bem definida, sem criar "ilhas de processamento útil em oceanos de código de checagem de erros". Essa clareza elimina código morto e resulta em arquiteturas muito mais simples e eficientes.

O Tratamento Disciplinado de Exceções

O documento traz uma redefinição crucial para o tratamento de erros em software. Meyer define uma "falha" como a incapacidade de uma rotina de cumprir o seu contrato, e uma "exceção" como a falha de uma sub-tarefa durante a execução dessa rotina.

Criticando linguagens como Ada e CLU, que utilizam exceções frequentemente como meras estruturas de controle que mascaram erros (por exemplo, retornar "zero" quando o cálculo de uma raiz quadrada falha, sem avisar o cliente), Meyer estabelece leis rígidas. Segundo ele, existem apenas duas reações válidas quando um contrato não pode ser cumprido:

Retomada (Resumption / Retry): A rotina restaura os objetos a um estado estável, corrige a condição que causou o erro, e tenta executar o algoritmo novamente.

Pânico Organizado (Organized Panic / Rescue): A rotina limpa a "bagunça" (restaurando o Invariante de Classe para deixar os objetos em um estado seguro), desiste de tentar cumprir o contrato e dispara uma nova exceção para reportar a falha ao cliente. A rotina jamais deve falhar silenciosamente.

Herança, Ligação Dinâmica e "Subcontratação"

No paradigma de Orientação a Objetos, a herança permite que uma classe filha redefina (redefine) comportamentos da classe pai. Meyer analisa isso sob a ótica dos contratos, classificando a redefinição como uma "subcontratação". Quando uma rotina é chamada utilizando ligação dinâmica, o trabalho é essencialmente delegado à versão da rotina específica daquele objeto no tempo de execução.

A regra fundamental para um "subcontratante honesto" é que ele não pode impor restrições maiores ao cliente do que o contrato original, nem pode entregar um resultado inferior. Isso se traduz na regra formal de redefinição: a nova rotina deve ter uma pré-condição igual ou mais fraca (aceitar mais casos) e uma pós-condição igual ou mais forte (garantir um resultado melhor ou mais específico). Isso evita o perigo de subclasses introduzirem comportamentos que quebrem as expectativas do software cliente. Meyer também alerta que a "ligação estática" em cenários polimórficos é um erro gravíssimo, pois pode aplicar lógicas antigas que violam os invariantes fortalecidos das novas classes filhas.

Conclusão e a Defesa das Funções Parciais

Ao concluir a obra, Meyer faz um apelo pragmático em defesa das "funções parciais", ou seja, rotinas que não são desenhadas para funcionar sob todo e

qualquer input possível. A busca ilusória por rotinas totais que consigam lidar com qualquer cenário imaginável resulta em sistemas inchados e frágeis.

O Design by Contract propõe um realismo estruturado: é muito melhor ter componentes humildes e parciais que estabelecem claramente, através de contratos rigorosos, o que farão e o que não farão. Ao transferir as amarras das especificações ambíguas para definições formais em nível de código (asserções que podem ser monitoradas e extraídas como documentação), Meyer mostra um caminho maduro, sistemático e altamente escalável para a engenharia de software. A obra mostra a linguagem Eiffel, estabelecendo princípios fundamentais que moldaram a compreensão moderna sobre qualidade, robustez e desenvolvimento orientado a objetos.